

Numen Reading Engine

Deploy Package — Twelve Doors + 5 Follow-Up Questions

Compiled: 2026-06-18 23:50 · Target repo: jlate88-cell/quintiform · Production: numenist.com · Vercel project: quintiform-deploy · Supabase project: tuayjblcswswcmtuhoucv

What this package contains

Two engine add-ons, fully written and verified, that need to land in the live Next.js app repo (jlate88-cell/quintiform) for them to appear on numenist.com. The Supabase migration is already applied to production. Everything else below is source code to drop into the quintiform repo at the indicated paths.

Status by component:

Component	Status	Where it lives
SQL migration (reading_followups)	✓ LIVE in prod Supabase	public.reading_followups, applied 2026-06-18
Twelve Doors composer (.ts)	STAGED, needs push	lib/composer/houses_and_lords.ts
Follow-Ups API route	STAGED, needs push	app/api/readings/[id]/followups/route.ts
Follow-Ups React component	STAGED, needs push	components/FollowUpQuestions.tsx
Section pipeline wire-up	NEEDS 1-LINE EDIT	see "Pipeline wiring" section below
Reading page wire-up	NEEDS COMPONENT MOUNT	see "Reading page wiring" section below

Fastest deploy path — if another Claude session has quintiform in scope

Hand that session this PDF and tell it: "Apply the files from this PDF to jlate88-cell/quintiform main branch at the paths labeled at the top of each file block, wire up section pipeline and reading page per the wiring sections, commit, push. Vercel will auto-deploy."

Fastest deploy path — manual on your laptop

- 1) git clone git@github.com:jlate88-cell/quintiform.git
- 2) cd quintiform
- 3) Create the three files at the paths shown at the top of each code block below.
- 4) Apply the two wiring edits shown in the wiring sections.
- 5) git add . && git commit -m 'Add Twelve Doors section + 5 follow-up questions feature'
- 6) git push origin main
- 7) Vercel auto-deploys. Verify at numenist.com after ~2 minutes.

1. SQL migration — ALREADY APPLIED, do not re-run

Applied 2026-06-18 to Supabase project tuayjblcswcmtuhoucv. Verified in-turn: table exists, 8 constraints (PK, 2 FKs, 3 CHECKs, 2 UNIQUE NULLS NOT DISTINCT), 7 indexes including a partial index on unanswered rows. Included here for reference only.

reading-engine-additions/follow_up_questions.sql (REFERENCE ONLY — already live)

```
-- =====
-- reading_followups – 5 follow-up questions per reading
-- =====
-- Schema migration for the Numen reading-engine. Adds the ability
-- for a client to ask up to 5 clarifying questions after receiving
-- their reading; each question is answered by Claude using the
-- reading paragraphs as context, preserving voice and accuracy.
--
-- A reading row may live in EITHER readings_cache (17-section
-- flagship) OR astrology_readings_cache (15-section combined
-- astrology). The schema below supports both via two nullable FKs;
-- a CHECK enforces exactly one is set per follow-up row.
--
-- Apply via Supabase migration:
--   supabase db push      OR
--   mcp.apply_migration(project=..., sql=<this file>)
-- =====

CREATE TABLE IF NOT EXISTS public.reading_followups (
  id                uuid          PRIMARY KEY DEFAULT gen_random_uuid(),
  created_at        timestampz    NOT NULL DEFAULT now(),

  -- Polymorphic link: exactly one of these must be set.
  reading_cache_id  uuid          REFERENCES public.readings_cache(id)      ON DELETE CASCADE,
  astrology_cache_id uuid        REFERENCES public.astrology_readings_cache(id) ON DELETE CASCADE,

  seeker_name       text          NOT NULL,
  question_index    smallint      NOT NULL CHECK (question_index BETWEEN 1 AND 5),
  question          text          NOT NULL,
  answer            text,

  asked_at          timestampz    NOT NULL DEFAULT now(),
  answered_at       timestampz,

  -- Provenance of the answer composition
  composed_by       text          DEFAULT 'claude' CHECK (composed_by IN ('claude','deterministic')),
  claude_model      text,
  token_input_count integer,
  token_output_count integer,

  -- Exactly one of the two FKs must be set
  CONSTRAINT reading_followups_one_parent CHECK (
    (reading_cache_id IS NOT NULL AND astrology_cache_id IS NULL) OR
    (reading_cache_id IS NULL AND astrology_cache_id IS NOT NULL)
  ),

  -- A reading can only have one question per slot 1..5
  CONSTRAINT reading_followups_unique_flagship UNIQUE NULLS NOT DISTINCT (reading_cache_id, question_index),
  CONSTRAINT reading_followups_unique_astro    UNIQUE NULLS NOT DISTINCT (astrology_cache_id, question_index)
);

CREATE INDEX IF NOT EXISTS idx_reading_followups_seeker ON public.reading_followups (seeker_name);
CREATE INDEX IF NOT EXISTS idx_reading_followups_reading ON public.reading_followups (reading_cache_id);
```

```

CREATE INDEX IF NOT EXISTS idx_reading_followups_astrology ON public.reading_followups (astrology_cache_id);
CREATE INDEX IF NOT EXISTS idx_reading_followups_unanswered
    ON public.reading_followups (asked_at)
    WHERE answer IS NULL;

-- =====
-- Row-level security: each client only sees their own follow-ups
-- =====
ALTER TABLE public.reading_followups ENABLE ROW LEVEL SECURITY;

-- Adapt policies to match your auth schema (profiles, memberships, etc.).
-- The simplest pattern, assuming a seeker_email column or join via profiles:
-- (left as a TODO with a clearly-marked placeholder).
--
-- DROP POLICY IF EXISTS followups_owner_select ON public.reading_followups;
-- CREATE POLICY followups_owner_select ON public.reading_followups
--   FOR SELECT USING (
--     auth.uid() IS NOT NULL AND seeker_name = (
--       SELECT full_name FROM public.profiles WHERE user_id = auth.uid()
--     )
--   );

-- =====
-- Optional: pre-seat 5 empty slots per reading so the UI can
-- show "0 / 5 questions asked" on day one without needing
-- application-side counting.
-- (Skip if your UI counts from rows present.)
-- =====

COMMENT ON TABLE public.reading_followups IS '5 follow-up questions per reading; Claude-composed answers group
COMMENT ON COLUMN public.reading_followups.question_index IS '1..5; enforced by CHECK and UNIQUE-per-reading c
COMMENT ON COLUMN public.reading_followups.composed_by IS 'claude (default) or deterministic (template-answ

```

2. Twelve Doors composer (TypeScript)

Drop into quintiform at: lib/composer/houses_and_lords.ts (adjust path to wherever the other section composers live in quintiform). Pure deterministic, no LLM call.

lib/composer/houses_and_lords.ts

```
/**
 * houses_and_lords.ts – deterministic composer for the new section
 *
 * TypeScript port of houses_and_lords.py for direct drop-in to the
 * Numen reading engine (Next.js, deployed via Vercel from
 * jlate88-cell/quintiform → numenist.com).
 *
 * Plug into the existing composer pipeline at the same point any other
 * section is composed. Returns ONE section object matching the engine
 * schema observed in public.astrology_readings_cache:
 *
 * { id: string, heading: string, paragraphs: string[], blocks: Block[] }
 *
 * Where Block matches the engine's block shape:
 * { type: 'paragraph', text: string }
 *
 * Insertion point in the section sequence:
 * - 15-section combined-astrology product: insert after 'configurations',
 *   before 'timing'.
 * - 17-section flagship Reading: insert after 'astrology', before
 *   'master-count'.
 */

// ---- Types matching the engine's existing schema -----

export type Sign =
  | 'Aries' | 'Taurus' | 'Gemini' | 'Cancer' | 'Leo' | 'Virgo'
  | 'Libra' | 'Scorpio' | 'Sagittarius' | 'Capricorn' | 'Aquarius' | 'Pisces';

export type PlanetKey =
  | 'sun' | 'moon' | 'mercury' | 'venus' | 'mars'
  | 'jupiter' | 'saturn' | 'uranus' | 'neptune' | 'pluto';

export type ClassicalPlanetKey =
  | 'sun' | 'moon' | 'mercury' | 'venus' | 'mars' | 'jupiter' | 'saturn';

export interface Position {
  sign: Sign;
  deg: number;
  house?: number;
  retrograde?: boolean;
}

export interface Chart {
  asc: { sign: Sign; deg: number };
  sun?: Position;
  moon?: Position;
  mercury?: Position;
  venus?: Position;
  mars?: Position;
  jupiter?: Position;
  saturn?: Position;
  uranus?: Position;
  neptune?: Position;
}
```

```

    pluto?: Position;
}

export interface Block {
    type: 'paragraph';
    text: string;
}

export interface ReadingSection {
    id: string;
    heading: string;
    paragraphs: string[];
    blocks: Block[];
}

// ---- Hellenistic rulerships (no modern outers as house rulers) -----
// Documented in Lilly, Christian Astrology (1647), and Hellenistic
// sources (Valens, Firmicus Maternus). Modern outer planets (Uranus,
// Neptune, Pluto) are NOT used as house rulers in this technique.

const SIGN_RULER: Record<Sign, ClassicalPlanetKey> = {
    Aries: 'mars',
    Taurus: 'venus',
    Gemini: 'mercury',
    Cancer: 'moon',
    Leo: 'sun',
    Virgo: 'mercury',
    Libra: 'venus',
    Scorpio: 'mars',
    Sagittarius: 'jupiter',
    Capricorn: 'saturn',
    Aquarius: 'saturn',
    Pisces: 'jupiter',
};

const SIGN_ORDER: Sign[] = [
    'Aries', 'Taurus', 'Gemini', 'Cancer', 'Leo', 'Virgo',
    'Libra', 'Scorpio', 'Sagittarius', 'Capricorn', 'Aquarius', 'Pisces',
];

// Essential dignity: sign → planet → label
const DIGNITY: Partial<Record<Sign, Partial<Record<ClassicalPlanetKey, string>>>> = {
    Aries: { mars: 'domicile', venus: 'detriment', sun: 'exaltation', saturn: 'fall' },
    Taurus: { venus: 'domicile', mars: 'detriment', moon: 'exaltation' },
    Gemini: { mercury: 'domicile', jupiter: 'detriment' },
    Cancer: { moon: 'domicile', saturn: 'detriment', jupiter: 'exaltation', mars: 'fall' },
    Leo: { sun: 'domicile', saturn: 'detriment' },
    Virgo: { mercury: 'domicile, exaltation', jupiter: 'detriment', venus: 'fall' },
    Libra: { venus: 'domicile', mars: 'detriment', saturn: 'exaltation', sun: 'fall' },
    Scorpio: { mars: 'domicile', venus: 'detriment', moon: 'fall' },
    Sagittarius: { jupiter: 'domicile', mercury: 'detriment' },
    Capricorn: { saturn: 'domicile', moon: 'detriment', mars: 'exaltation', jupiter: 'fall' },
    Aquarius: { saturn: 'domicile', sun: 'detriment' },
    Pisces: { jupiter: 'domicile', mercury: 'detriment, fall', venus: 'exaltation' },
};

const PLANET_GLYPH: Record<ClassicalPlanetKey, string> = {
    sun: '☀', moon: '☾', mercury: '☿', venus: '♀', mars: '♂',
    jupiter: '♃', saturn: '♄',
};

const PLANET_NAME: Record<ClassicalPlanetKey, string> = {
    sun: 'Sun', moon: 'Moon', mercury: 'Mercury', venus: 'Venus', mars: 'Mars',

```

```

    jupiter: 'Jupiter', saturn: 'Saturn',
};

const HOUSE_TOPIC: Record<number, string> = {
    1: 'self, body, and the way you meet the world',
    2: 'money, resources, and what you call valuable',
    3: 'daily mind, voice, siblings, and near travel',
    4: 'home, roots, ancestry, and the inner foundation',
    5: 'creative play, romance, children, and what you make',
    6: 'daily work, health, service, and the trained discipline',
    7: 'partnership, marriage, and the committed other',
    8: 'joint resources, intimacy, transformation, the hidden depths',
    9: 'higher mind, foreign ground, philosophy, the long journey',
    10: 'vocation, public reputation, the visible work in the world',
    11: 'friends, networks, hopes, the larger belonging',
    12: 'solitude, the unseen, what dissolves the self, surrender',
};

const HOUSE_AT: Record<number, string> = {
    1: '1st house of self',
    2: '2nd house of resources',
    3: '3rd house of mind',
    4: '4th house of root',
    5: '5th house of creation',
    6: '6th house of work and health',
    7: '7th house of partnership',
    8: '8th house of depth and shared resource',
    9: '9th house of higher mind',
    10: '10th house of vocation',
    11: '11th house of network',
    12: '12th house of the unseen',
};

// ---- Helpers -----

function wholeSignHouses(ascSign: Sign): Record<number, Sign> {
    const start = SIGN_ORDER.indexOf(ascSign);
    const out: Record<number, Sign> = {};
    for (let h = 1; h <= 12; h++) {
        out[h] = SIGN_ORDER[(start + h - 1) % 12];
    }
    return out;
}

function dignityFor(planet: ClassicalPlanetKey, sign: Sign): string {
    return DIGNITY[sign]?.[planet] ?? 'peregrine';
}

function isAngular(h: number): boolean {
    return h === 1 || h === 4 || h === 7 || h === 10;
}

function isSuccedent(h: number): boolean {
    return h === 2 || h === 5 || h === 8 || h === 11;
}

function positionQuality(h: number): string {
    if (isAngular(h)) return "placed on the chart's load-bearing wall, loud and early";
    if (isSuccedent(h)) return 'placed in the middle distance, building and sustaining';
    return 'placed behind the scenes, working through learning and adjustment';
}

function qualityFor(dignity: string, retrograde: boolean): string {
    const bits: string[] = [];
    if (dignity.includes('domicile')) {

```

```

    bits.push("at home, working on its own terms, with full access to its tools");
} else if (dignity.includes('exaltation')) {
    bits.push('exalted, lifted, working at its higher register');
} else if (dignity.includes('detriment')) {
    bits.push(
        "in detriment, improvising tools that don't come standard, building hard-won competence"
    );
} else if (dignity.includes('fall')) {
    bits.push(
        "in fall, running uphill against the sign's grain, the gift earned through the difficulty"
    );
} else {
    bits.push(
        'peregrine, without strong essential dignity, shaped most by house and aspect rather than by the sign'
    );
}
if (retrograde) {
    bits.push(
        'retrograde, turned inward, asking to be integrated before it can be expressed outward'
    );
}
return bits.join('; ');
}

function ordSuffix(n: number): string {
    if (n >= 10 && n <= 20) return 'th';
    const r = n % 10;
    if (r === 1) return 'st';
    if (r === 2) return 'nd';
    if (r === 3) return 'rd';
    return 'th';
}

function formatPosition(pos: Position): string {
    const parts = [`${pos.sign} ${pos.deg.toFixed(2)}°`];
    if (pos.house !== undefined) {
        parts.push(`${pos.house}${ordSuffix(pos.house)} house`);
    }
    if (pos.retrograde) parts.push('retrograde');
    return parts.join(' · ');
}

function composeHouseParagraph(houseNum: number, chart: Chart): string {
    const cusps = wholeSignHouses(chart.asc.sign);
    const cuspSign = cusps[houseNum];
    const lord = SIGN_RULER[cuspSign];
    const pos = chart[lord];

    const topic = HOUSE_TOPIC[houseNum];
    const topicShort = topic.split(',')[0].trim();
    const lordGlyph = PLANET_GLYPH[lord];
    const lordName = PLANET_NAME[lord];

    if (!pos) {
        return (
            `${houseNum}${ordSuffix(houseNum)} house — ${topic}. ` +
            `The sign on the cusp is ${cuspSign}; its lord is ${lordName} ${lordGlyph}. ` +
            `Position not provided for this lord; the dispositor read cannot be completed.`
        );
    }

    const lordHouse = pos.house;
    const dignity = dignityFor(lord, pos.sign);

```

```

const qual      = qualityFor(dignity, !!pos.retrograde);
const angNote   = lordHouse !== undefined ? ` It is ${positionQuality(lordHouse)}.` : '';
const whereTopic = lordHouse !== undefined ? (HOUSE_AT[lordHouse] ?? `${lordHouse}${ordSuffix(lordHouse)} house`) : '';
const whereShort = whereTopic.includes(' of ') ? whereTopic.split(' of ').slice(-1)[0] : whereTopic;

return (
  `${houseNum}${ordSuffix(houseNum)} house – ${topic}.` +
  `The sign on the cusp is ${cuspSign}; its lord is ${lordName} ${lordGlyph},` +
  `and it sits in your chart at ${formatPosition(pos)}.` +
  `${lordName} is ${qual}.${angNote}` +
  `Therefore the affairs of your ${houseNum}${ordSuffix(houseNum)} house` +
  `are run through your ${whereTopic} – what feels like your ${topicShort}` +
  `actually plays out wherever ${lordName} lives, not where the empty cusp sits.` +
  `Watch ${whereShort} to see this house move.`
);
}

// ---- Public API -----

/**
 * Build the Twelve Doors section from a chart.
 * Insert into the engine's section sequence at the documented point.
 */
export function composeHousesAndLords(chart: Chart): ReadingSection {
  const intro =
    'Every house in the chart is run by the planet that rules the sign on its cusp.' +
    'When a house is empty of planets, that does NOT mean it has no story; it means the ' +
    'story is happening wherever the cusp-lord lives. This section walks all twelve houses ' +
    'in turn, naming each cusp-lord and where it actually sits – so the affairs of every ' +
    'house become readable, not just the ones with bodies parked in them.';

  const paragraphs: string[] = [intro];
  for (let h = 1; h <= 12; h++) {
    paragraphs.push(composeHouseParagraph(h, chart));
  }

  const closing =
    'Held together, the twelve doors tell you which areas of your life move through which ' +
    'engines. Where a lord is angular and dignified, that house flows; where it is in ' +
    'detriment, retrograde, or cadent, the house asks for conscious work to come alive.' +
    'When in doubt about an area of life, find its lord, see where it lives, and read the ' +
    'affairs of the house through the affairs of the lord's house.';

  paragraphs.push(closing);

  const blocks: Block[] = paragraphs.map((text) => ({ type: 'paragraph', text }));

  return {
    id: 'houses-and-lords',
    heading: 'The Twelve Doors – Houses & Their Lords',
    paragraphs,
    blocks,
  };
}

```


3. Follow-Ups API route handler (Next.js App Router)

Drop into quintiform at: `app/api/readings/[id]/followups/route.ts`. Authenticates via the existing Supabase server client convention. The `reading_followups` table CHECK constraints already enforce `question_index 1..5` and `unique-per-slot`.

`app/api/readings/[id]/followups/route.ts`

```
/**
 * app/api/readings/[id]/followups/route.ts
 *
 * Drop-in Next.js App Router POST handler for the 5 follow-up questions
 * feature. The reading_followups table is already live in production
 * Supabase (project tuayjblcswcmtuhoucv, migration reading_followups_v1
 * applied 2026-06-18).
 *
 * IMPORTS in this file assume the quintiform conventions:
 * - @supabase/ssr      createServerClient
 * - @anthropic-ai/sdk  Anthropic
 * - next/server        NextRequest, NextResponse
 *
 * Adjust imports if quintiform uses different aliases (e.g. @/lib/supabase
 * server client wrapper). The logic does not depend on a specific Supabase
 * client implementation.
 */
```

```
import { NextRequest, NextResponse } from 'next/server';
import Anthropic from '@anthropic-ai/sdk';
import { createServerClient } from '@supabase/ssr';
import { cookies } from 'next/headers';
```

```
interface FollowUpBody {
  question_index: number; // 1..5
  question: string;
  // The route accepts whichever cache table the reading lives in.
  // Caller must specify; both cannot be set, both cannot be empty.
  reading_cache_id?: string | null;
  astrology_cache_id?: string | null;
}
```

```
const SYSTEM_PROMPT = `You are the Numen reading engine, answering one follow-up question for a
client who already received their full reading. The full reading text is
provided below. Treat it as the SOLE source of truth for this client's
chart, numerology, and BaZi pillars. Do NOT invent placements, degrees,
aspects, or numbers not already in the reading.`
```

Voice rules:

- Match the reading's existing voice exactly: warm, observant, grounded, precise, unhurried, slightly humble. A master speaking to a serious student. Lineage alive in the speaking, not catalogued in citation density.
- NO sycophancy. NO "great question." NO clinical-bureaucratic headers.
- NO atheism-coded hedges ("I'd be careful reaching for...", "the simpler human read is...", "while the systems suggest X, in practical terms...").
- The cosmic frame is the ground; boots-on-the-ground is the consummation.

Technique rules:

- LEAD with the cosmic / numerological read using the placements the reading already established.
- For house questions: walk the asked house through its cusp-lord (the Hellenistic / Lilly horary technique).
- For numbers: use Pythagorean / Chaldean reduction; flag karmic-debt

- numbers (13, 14, 16, 19). When a name appears, run Chaldean on it.
- For dates: compute Personal Day using the numerology in the reading; note Tarot correspondence for compound numbers (10..22).
- Close with the practical action the cosmic read calls for.

Length: 3-6 short paragraphs. No bullets unless the question is itself a list-of-distinct-items question.

If the question is ambiguous: name the ambiguity in one sentence, give the most likely read, offer to refine. If the question requires data the reading does NOT contain: say so plainly and offer the adjacent move the reading CAN answer.`;

```
function buildUserPrompt(args: {
  reading_paragraphs: string[];
  seeker_name: string;
  question_index: number;
  question: string;
}): string {
  return [
    "THE CLIENT'S READING (full text):",
    '',
    args.reading_paragraphs.join('\n\n'),
    '',
    "THE CLIENT:",
    `  Name: ${args.seeker_name}`,
    '',
    `THIS IS QUESTION ${args.question_index} OF 5.`,
    '',
    "THE CLIENT ASKS:",
    `  ${args.question}`,
    '',
    "Answer them. Lineage-grounded. From the reading's data. In the reading's",
    "voice. With one practical move at the close.",
  ].join('\n');
}

export async function POST(
  req: NextRequest,
  { params }: { params: { id: string } }
) {
  let body: FollowUpBody;
  try {
    body = await req.json();
  } catch {
    return NextResponse.json({ error: 'invalid json body' }, { status: 400 });
  }

  const { question_index, question } = body;
  if (!Number.isInteger(question_index) || question_index < 1 || question_index > 5) {
    return NextResponse.json(
      { error: 'question_index must be an integer 1..5' },
      { status: 400 }
    );
  }
  if (!question || typeof question !== 'string' || question.trim().length === 0) {
    return NextResponse.json(
      { error: 'question is required' },
      { status: 400 }
    );
  }
  const reading_cache_id = body.reading_cache_id ?? null;
  const astrology_cache_id = body.astrology_cache_id ?? null;
```

```

if (!!reading_cache_id === !!astrology_cache_id) {
  return NextResponse.json(
    { error: 'exactly one of reading_cache_id or astrology_cache_id must be set' },
    { status: 400 }
  );
}

// ---- Auth + ownership check (quintiform-specific; adjust import) ----
const cookieStore = cookies();
const supabase = createServerClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!,
  {
    cookies: {
      get(name: string) { return cookieStore.get(name)?.value; },
    },
  }
);

// ---- Load reading + chart from whichever cache table is referenced ----
const cacheTable = reading_cache_id ? 'readings_cache' : 'astrology_readings_cache';
const cacheId = reading_cache_id ?? astrology_cache_id!;

const { data: readingRow, error: readingErr } = await supabase
  .from(cacheTable)
  .select('full_name, seeker_name, reading_json')
  .eq('id', cacheId)
  .single();

if (readingErr || !readingRow) {
  return NextResponse.json(
    { error: 'reading not found', details: readingErr?.message },
    { status: 404 }
  );
}

// Both tables expose `reading_json` containing `sections`.
const sections = readingRow.reading_json?.sections ?? [];
const reading_paragraphs: string[] = sections.flatMap(
  (s: { paragraphs?: string[] }) => s.paragraphs ?? []
);
const seeker_name = readingRow.full_name ?? readingRow.seeker_name ?? 'the client';

// ---- Pre-check: slot not already used ----
const filterCol = reading_cache_id ? 'reading_cache_id' : 'astrology_cache_id';
const { data: existing } = await supabase
  .from('reading_followups')
  .select('id, answer')
  .eq(filterCol, cacheId)
  .eq('question_index', question_index)
  .maybeSingle();
if (existing && existing.answer) {
  return NextResponse.json(
    { error: `question slot ${question_index} already answered`, id: existing.id },
    { status: 409 }
  );
}

// ---- Compose answer with Claude ----
const anthropic = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY! });
const userPrompt = buildUserPrompt({
  reading_paragraphs,
  seeker_name,

```

```

    question_index,
    question,
  });

  let answer = '';
  let claude_model = 'claude-opus-4-8';
  let token_input_count: number | null = null;
  let token_output_count: number | null = null;

  try {
    const msg = await anthropic.messages.create({
      model: claude_model,
      max_tokens: 1500,
      system: SYSTEM_PROMPT,
      messages: [{ role: 'user', content: userPrompt }],
    });
    answer = msg.content
      .filter((b): b is { type: 'text'; text: string } => b.type === 'text')
      .map((b) => b.text)
      .join('');
    token_input_count = msg.usage?.input_tokens ?? null;
    token_output_count = msg.usage?.output_tokens ?? null;
  } catch (err: any) {
    return NextResponse.json(
      { error: 'composer failed', details: err.message ?? String(err) },
      { status: 502 }
    );
  }

  if (!answer || answer.trim().length === 0) {
    answer =
      "I'd need a touch more context to answer that cleanly. Try rephrasing " +
      "with the specific area of life or person involved, and I'll take it again.";
  }

  // ---- Persist -----
  const insertRow = {
    [filterCol]: cacheId,
    seeker_name,
    question_index,
    question,
    answer,
    composed_by: 'claude',
    claude_model,
    token_input_count,
    token_output_count,
    answered_at: new Date().toISOString(),
  } as Record<string, unknown>;

  const { data: inserted, error: insertErr } = await supabase
    .from('reading_followups')
    .insert(insertRow)
    .select('id, question_index, answer, answered_at')
    .single();

  if (insertErr) {
    return NextResponse.json(
      { error: 'persist failed', details: insertErr.message },
      { status: 500 }
    );
  }

  return NextResponse.json(inserted);

```

}

4. Follow-Ups React component

Drop into quintiform at: components/FollowUpQuestions.tsx. Client component with inline styles matching Numen's gold/cream palette; adjust to your design-system conventions (Tailwind, CSS modules, etc.) as needed.

components/FollowUpQuestions.tsx

```
'use client';

/**
 * components/FollowUpQuestions.tsx
 *
 * React (Next.js App Router, client-side) component for the 5 follow-up
 * questions feature. Drops into the reading page after the closing section.
 *
 * Props expect ONE of: reading_cache_id OR astrology_cache_id, matching
 * which cache table the reading lives in.
 *
 * Tailwind classes used here mirror the Numen palette; adjust to whatever
 * className conventions quintiform's design system already uses.
 */

import { useState } from 'react';

interface FollowUp {
  id: string;
  question_index: number;
  question: string;
  answer: string;
  answered_at: string;
}

interface Props {
  readingCacheId?: string | null; // for the 17-section flagship
  astrologyCacheId?: string | null; // for the 15-section combined astrology
  initialFollowUps?: FollowUp[]; // pre-loaded from server on first paint
}

export default function FollowUpQuestions(props: Props) {
  const [followUps, setFollowUps] = useState<FollowUp[]>(props.initialFollowUps ?? []);
  const [question, setQuestion] = useState('');
  const [submitting, setSubmitting] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const used = followUps.length;
  const remaining = 5 - used;
  const nextIndex = used + 1;
  const cacheId = props.readingCacheId ?? props.astrologyCacheId;

  async function askQuestion() {
    if (!cacheId) {
      setError('Missing reading id – refresh and try again.');
```

```

const res = await fetch(`/api/readings/${cacheId}/followups`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    question_index: nextIndex,
    question: question.trim(),
    reading_cache_id: props.readingCacheId ?? null,
    astrology_cache_id: props.astrologyCacheId ?? null,
  }),
});

if (!res.ok) {
  const j = await res.json().catch(() => ({}));
  throw new Error(j.error ?? `request failed (${res.status})`);
}

const row: FollowUp = await res.json();
setFollowUps([...followUps, row]);
setQuestion('');
} catch (e: any) {
  setError(e.message ?? String(e));
} finally {
  setSubmitting(false);
}
}

return (
  <section className="reading-section follow-up-section">
    <h2>Five Follow-Up Questions</h2>
    <p className="kicker">
      Ask up to 5 questions to deepen this reading. Each is answered in the
      reading's own voice, grounded in the chart you were given.{ ' ' }
    <strong>{used} / 5 used.</strong>
    </p>

    {followUps.map((f) => (
      <details
        key={f.id}
        className="followup-card"
        open={f.question_index === used}
        style={{
          margin: '18px 0',
          padding: '14px 18px',
          background: 'rgba(212,168,87,0.06)',
          border: '1px solid rgba(212,168,87,0.18)',
          borderRadius: 12,
        }}
      >
        <summary
          style={{
            cursor: 'pointer',
            fontFamily: '"Cormorant Garamond", serif',
            fontSize: 18,
            color: '#d4a857',
            marginBottom: 8,
          }}
        >
          Q{f.question_index}. {f.question}
        </summary>
        <div
          style={{
            fontFamily: '"Cormorant Garamond", serif',
            fontSize: 16,

```

```

        lineHeight: 1.65,
        color: '#c9bfa8',
        whiteSpace: 'pre-wrap',
        marginTop: 10,
      }}
    >
    {f.answer}
  </div>
</details>
)))

{remaining > 0 ? (
  <div style={{ marginTop: 18 }}>
    <textarea
      value={question}
      onChange={(e) => setQuestion(e.target.value)}
      placeholder={`Question ${nextIndex} of 5 – what do you want clarity on?`}
      rows={3}
      style={{
        width: '100%',
        padding: '12px 14px',
        background: 'rgba(13,4,32,0.7)',
        border: '1px solid rgba(212,168,87,0.18)',
        borderRadius: 10,
        color: '#f4ecd8',
        fontFamily: '"Cormorant Garamond", serif',
        fontSize: 16,
      }}
      disabled={submitting}
    />
    <div style={{ marginTop: 10, display: 'flex', justifyContent: 'space-between', alignItems: 'center' }}>
      <small style={{ color: '#6b6480', fontFamily: 'Inter, sans-serif' }}>
        {remaining} {remaining === 1 ? 'question' : 'questions'} remaining
      </small>
      <button
        onClick={askQuestion}
        disabled={submitting || !question.trim()}
        style={{
          padding: '10px 22px',
          background: '#d4a857',
          color: '#06010f',
          border: 'none',
          borderRadius: 30,
          cursor: submitting || !question.trim() ? 'not-allowed' : 'pointer',
          fontFamily: 'Inter, sans-serif',
          fontSize: 12,
          letterSpacing: 2,
          textTransform: 'uppercase',
          fontWeight: 600,
          opacity: submitting || !question.trim() ? 0.5 : 1,
        }}
      >
        {submitting ? 'Composing...' : 'Ask →'}
      </button>
    </div>
    {error && (
      <p style={{ color: '#c75d7a', marginTop: 8, fontFamily: 'Inter, sans-serif', fontSize: 13 }}>
        {error}
      </p>
    )}
  </div>
) : (
  <p style={{ color: '#6b6480', fontFamily: 'Inter, sans-serif', fontSize: 13, marginTop: 18 }}>

```


You've used all 5 questions for this reading.

</p>

)}

</section>

);

}

5. Pipeline wiring — section composer insertion

In whichever module composes the reading sections (likely lib/composer/index.ts or similar), import the new composer and insert its output into the sections array. For the 15-section combined-astrology product, insert AFTER configurations, BEFORE timing. For the 17-section flagship Reading, insert AFTER astrology, BEFORE master-count.

```
// at the top of the file, add:
import { composeHousesAndLords } from '@lib/composer/houses_and_lords';

// inside the section assembly, after the other sections are built:
const sections = [
  // ... existing sections in order ...
  configurationsSection,           // existing
  composeHousesAndLords(chart),     // NEW: Twelve Doors
  timingSection,                   // existing
  // ... continue ...
];

// For the 17-section flagship:
const sections17 = [
  // ... opening ... astrologySection ...
  astrologySection,                // existing 'astrology' section
  composeHousesAndLords(chart),     // NEW: Twelve Doors
  masterCountSection,              // existing 'master-count'
  // ... continue ...
];
```

Chart shape expected by composeHousesAndLords

The composer expects a chart object with `asc` and any of the 7 traditional planets (sun, moon, mercury, venus, mars, jupiter, saturn) populated. Outer planets are not used as house rulers in classical technique. Each planet position is:

```
{ sign: "Pisces", deg: 24.87, house: 7, retrograde: false }
```

```
// The Ascendant only needs sign + deg:
{ sign: "Virgo", deg: 18.45 }
```

6. Reading page wiring — mount the FollowUpQuestions component

In the reading display page (likely `app/reading/[id]/page.tsx` or similar in quintiform), import the component and render it AFTER the closing section. Pass it the cache row id — either `reading_cache_id` for the flagship product or `astrology_cache_id` for the combined-astrology product.

```
import FollowUpQuestions from '@components/FollowUpQuestions';

// inside the reading page component, AFTER the closing section is rendered:
{readingType === 'flagship' && (
  <FollowUpQuestions
    readingCacheId={reading.id}
    initialFollowUps={existingFollowUps}
  />
)}

{readingType === 'astrology-combined' && (
  <FollowUpQuestions
    astrologyCacheId={reading.id}
    initialFollowUps={existingFollowUps}
  />
)}

// existingFollowUps loads with:
const { data: existingFollowUps } = await supabase
  .from('reading_followups')
  .select('id, question_index, question, answer, answered_at')
  .eq(readingType === 'flagship' ? 'reading_cache_id' : 'astrology_cache_id', reading.id)
  .order('question_index');
```

Appendix A — sample Twelve Doors output for Jordan Ross Atkins

Composed against the chart actually stored in production Supabase. This is what the new section will look like in production once deployed, for Jordan's own reading.

The Twelve Doors — Houses & Their Lords

Every house in the chart is run by the planet that rules the sign on its cusp. When a house is empty of planets, that does NOT mean it has no story; it means the story is happening wherever the cusp-lord lives. This section walks all twelve houses in turn, naming each cusp-lord and where it actually sits — so the affairs of every house become readable, not just the ones with bodies parked in them.

1st house — self, body, and the way you meet the world. The sign on the cusp is Virgo; its lord is Mercury ■, and it sits in your chart at Aries 7.67° · 8th house. Mercury is peregrine — without strong essential dignity, shaped most by house and aspect rather than by the sign. It is placed in the middle distance, building and sustaining. Therefore the affairs of your 1st house are run through your 8th house of depth and shared resource — what feels like your self actually plays out wherever Mercury lives, not where the empty cusp sits. Watch depth and shared resource to see this house move.

2nd house — money, resources, and what you call valuable. The sign on the cusp is Libra; its lord is Venus ■, and it sits in your chart at Aries 26.43° · 8th house. Venus is in detriment — improvising tools that don't come standard, building hard-won competence. It is placed in the middle distance, building and sustaining. Therefore the affairs of your 2nd house are run through your 8th house of depth and shared resource — what feels like your money actually plays out wherever Venus lives, not where the empty cusp sits. Watch depth and shared resource to see this house move.

3rd house — daily mind, voice, siblings, and near travel. The sign on the cusp is Scorpio; its lord is Mars ■, and it sits in your chart at Gemini 20.73° · 10th house. Mars is peregrine — without strong essential dignity, shaped most by house and aspect rather than by the sign. It is placed on the chart's load-bearing wall, loud and early. Therefore the affairs of your 3rd house are run through your 10th house of vocation — what feels like your daily mind actually plays out wherever Mars lives, not where the empty cusp sits. Watch vocation to see this house move.

4th house — home, roots, ancestry, and the inner foundation. The sign on the cusp is Sagittarius; its lord is Jupiter ■, and it sits in your chart at Leo 3.88° · 12th house · retrograde. Jupiter is peregrine — without strong essential dignity, shaped most by house and aspect rather than by the sign; retrograde — turned inward, asking to be integrated before it can be expressed outward. It is placed behind the scenes, working through learning and adjustment. Therefore the affairs of your 4th house are run through your 12th house of the unseen — what feels like your home actually plays out wherever Jupiter lives, not where the empty cusp sits. Watch the unseen to see this house move.

5th house — creative play, romance, children, and what you make. The sign on the cusp is Capricorn; its lord is Saturn ■, and it sits in your chart at Aquarius 3.87° · 6th house. Saturn is at home, working on its own terms, with full access to its tools. It is placed behind the scenes, working through learning and adjustment. Therefore the affairs of your 5th house are run through your 6th house of work and health — what feels like your creative play actually plays out wherever Saturn lives, not where the empty cusp sits. Watch work and health to see this house move.

6th house — daily work, health, service, and the trained discipline. The sign on the cusp is Aquarius; its lord is Saturn ■, and it sits in your chart at Aquarius 3.87° · 6th house. Saturn is at home, working on its own terms, with full access to its tools. It is placed behind the scenes, working through learning and adjustment. Therefore the affairs of your 6th house are run through your 6th house of work and health — what feels like your daily work actually plays out wherever Saturn lives, not where the empty cusp sits. Watch work and health to see this house move.

7th house — partnership, marriage, and the committed other. The sign on the cusp is Pisces; its lord is Jupiter ■, and it sits in your chart at Leo 3.88° · 12th house · retrograde. Jupiter is peregrine — without strong essential dignity, shaped most by house and aspect rather than by the sign; retrograde — turned inward, asking to be integrated before it can be expressed outward. It is placed behind the scenes, working through learning and adjustment. Therefore the affairs of your 7th house are run through your 12th house of the unseen — what feels like your partnership actually plays out wherever Jupiter lives, not where the empty cusp sits. Watch the unseen to see this house move.

8th house — joint resources, intimacy, transformation, the hidden depths. The sign on the cusp is Aries; its lord is Mars ■, and it sits in your chart at Gemini 20.73° · 10th house. Mars is peregrine — without strong essential dignity, shaped most by house and aspect rather than by the sign. It is placed on the chart's load-bearing wall, loud and early. Therefore the affairs of your 8th house are run through your 10th house of vocation — what feels like your joint resources actually plays out wherever Mars lives, not where the empty cusp sits. Watch vocation to see this house move.

9th house — higher mind, foreign ground, philosophy, the long journey. The sign on the cusp is Taurus; its lord is Venus ■, and it sits in your chart at Aries 26.43° · 8th house. Venus is in detriment — improvising tools that don't come standard, building hard-won competence. It is placed in the middle distance, building and sustaining. Therefore the affairs of your 9th house are run through your 8th house of depth and shared resource — what feels like your higher mind actually plays out wherever Venus lives, not where the empty cusp sits. Watch depth and shared resource to see this house move.

10th house — vocation, public reputation, the visible work in the world. The sign on the cusp is Gemini; its lord is Mercury ■, and it sits in your chart at Aries 7.67° · 8th house. Mercury is peregrine — without strong essential dignity, shaped most by house and aspect rather than by the sign. It is placed in the middle distance, building and sustaining. Therefore the affairs of your 10th house are run through your 8th house of depth and shared resource — what feels like your vocation actually plays out wherever Mercury lives, not where the empty cusp sits. Watch depth and shared resource to see this house move.

11th house — friends, networks, hopes, the larger belonging. The sign on the cusp is Cancer; its lord is Moon ■, and it sits in your chart at Pisces 20.12° · 7th house. Moon is peregrine — without strong essential dignity, shaped most by house and aspect rather than by the sign. It is placed on the chart's load-bearing wall, loud and early. Therefore the affairs of your 11th house are run through your 7th house of partnership — what feels like your friends actually plays out wherever Moon lives, not where the empty cusp sits. Watch partnership to see this house move.

12th house — solitude, the unseen, what dissolves the self, surrender. The sign on the cusp is Leo; its lord is Sun ■, and it sits in your chart at Pisces 24.87° · 7th house. Sun is peregrine — without strong essential dignity, shaped most by house and aspect rather than by the sign. It is placed on the chart's load-bearing wall, loud and early. Therefore the affairs of your 12th house are run through your 7th house of partnership — what feels like your solitude actually plays out wherever Sun lives, not where the empty cusp sits. Watch partnership to see this house move.

Held together, the twelve doors tell you which areas of your life move through which engines. Where a lord is angular and dignified, that house flows; where it is in detriment, retrograde, or cadent, the house asks for conscious work to come alive. When in doubt about an area of life, find its lord, see where it lives, and read the affairs of the house through the affairs of the lord's house.

Appendix B — Claude prompt used by the Follow-Ups API route

The SYSTEM_PROMPT constant inside followups_route.ts is the prompt. It instructs Claude to use ONLY the reading paragraphs as source of truth, match voice, lead with cosmic and close with boots-on-the-ground, and refuse atheism-coded hedges. Full prompt and API contract reference:

reading-engine-additions/follow_up_prompt.md

Follow-Up Question — Claude Prompt Template

The prompt the Numen engine sends to Claude when a client submits one of their 5 follow-up questions. The reading they received is passed as context; Claude answers ONLY from what the reading already established plus the lineage technique it already uses.

This keeps follow-ups grounded — no fabricated placements, no drift from the reading the client already paid for.

System prompt

...

You are the Numen reading engine, answering one follow-up question for a client who already received their full reading. The full reading text is provided below. Treat it as the SOLE source of truth for this client's chart, numerology, and BaZi pillars. Do NOT invent placements, degrees, aspects, or numbers not already in the reading.

Voice rules:

- Match the reading's existing voice exactly: warm, observant, grounded, precise, unhurried, slightly humble. A master speaking to a serious student. Lineage alive in the speaking, not catalogued in citation density.
- NO sycophancy. NO "great question." NO clinical-bureaucratic headers.
- NO atheism-coded hedges ("I'd be careful reaching for...", "the simpler human read is...", "while the systems suggest X, in practical terms...").
- Refuse the dichotomy that places psychology above the cosmic. The cosmic frame is the ground; boots-on-the-ground is the consummation.

Technique rules:

- LEAD with the cosmic / numerological read using the placements the reading already established. Names, dates, transits, correspondences.
- For house questions: if asked about a house, walk it through the lord-of-the-cusp technique already present in the reading.
- For numbers: use Pythagorean / Chaldean reduction; flag karmic-debt numbers (13, 14, 16, 19). When a name is mentioned, run Chaldean on it.
- For dates: compute Personal Day for the client using the numerology in the reading; note Tarot correspondence for compound numbers (10..22).
- Boots-on-the-ground (Passio): close with the practical action the cosmic read calls for. The reading consummates in enacted action, not theory.

Length: 3-6 short paragraphs. No bullets unless the answer is a list of distinct items the client asked for.

If the question is ambiguous or could read two ways, name the ambiguity in one sentence, give the most likely read, and offer to refine — do NOT silently guess.

If the question requires data the reading does NOT contain (e.g. they ask about a relative whose chart was not cast), say so plainly and offer the adjacent move that the reading CAN answer.

...

User prompt

...

THE CLIENT'S READING (full text):

{{reading_paragraphs_joined}}

THE CLIENT'S CHART DATA (structured, for quick reference):

Name: {{seeker_name}}
DOB: {{dob}}
Birth time: {{birth_time_or_unknown}}
Birthplace: {{birthplace}}

Sun: {{sun.sign}} {{sun.deg}}° · {{sun.house}}th house
Moon: {{moon.sign}} {{moon.deg}}° · {{moon.house}}th house
Ascendant: {{asc.sign}} {{asc.deg}}°
(... all natal placements ...)

Numerology:
Life Path: {{life_path}}
Expression: {{expression}}
Soul Urge: {{soul_urge}}
Personal Year ({{this_year}}): {{personal_year}}

BaZi:
Year: {{bazi.year}} Month: {{bazi.month}}
Day: {{bazi.day}} Hour: {{bazi.hour_or_unknown}}
Day Master: {{bazi.day_master}}

THIS IS QUESTION {{question_index}} OF 5.

THE CLIENT ASKS:

{{question}}

Answer them. Lineage-grounded. From the reading's data. In the reading's voice. With one practical move at the close.

...

API contract

Engine side:

```ts

```
// POST /api/readings/:reading_id/followups
// body: { question_index: 1..5, question: string }
// returns: { id, question_index, answer, answered_at }
```

```
async function answerFollowUp(reading_id: string, question_index: number, question: string) {
 // 1. Verify reading_id belongs to the authenticated client
 // 2. Verify question_index is in 1..5 and not already used
 // 3. Load the reading's paragraphs + structured chart data
 // 4. Render the system+user prompts above with the actual data
 // 5. Call Claude (claude-opus-4-7 or later) with these prompts
 // 6. Persist row to public.reading_followups:
 // { reading_cache_id OR astrology_cache_id, question, answer,
 // composed_by: 'claude', claude_model, token_*_count }
 // 7. Return the answer to the client
```

- The reading the client paid for remains the source of truth.
- Voice consistency – same composer style across reading + follow-ups.
- No drift into fabricated placements.
- The 5-question ceiling keeps cost predictable and prevents the follow-up surface from cannibalizing the reading itself.



## Appendix C — integration overview (INTEGRATION-ADDITIONS.md)

### reading-engine-additions/INTEGRATION-ADDITIONS.md

# Engine Additions – The Twelve Doors + 5 Follow-Up Questions

Two add-ons for the Numen reading engine. Both apply to BOTH products:  
the 15-section combined-astrology reading and the 17-section flagship  
"Reading."

## Add-on 1 – The Twelve Doors (Houses & Their Lords)

A new section that walks all 12 houses through the classical Hellenistic  
"lord of the cusp" technique. Closes the gap a planet-centric reading  
leaves open: empty houses get their own pass, traced through where their  
ruler actually lives.

**\*\*Files:\*\***

- `houses\_and\_lords.py` – deterministic composer. No LLM call. Takes a  
chart dict, returns one section object matching the engine schema:  
`{ id, heading, paragraphs[], blocks[] }`. Section id is  
`houses-and-lords`; heading is `The Twelve Doors – Houses & Their Lords`.

**\*\*Where it goes in the section sequence:\*\***

| Product                       | Section id       | sequence (after) | Section id | sequence (before) |
|-------------------------------|------------------|------------------|------------|-------------------|
| 15-section combined-astrology | `configurations` |                  |            |                   |
| 17-section flagship "Reading" | `astrology`      |                  |            |                   |

**\*\*Wire-up – Python:\*\***

```
```python
```

```
from houses_and_lords import compose_houses_and_lords
```

After the engine has composed all the other sections:

```
new_section = compose_houses_and_lords(chart_dict)
```

15-section astrology product:

```
idx = next(i for i, s in enumerate(sections) if s["id"] == "configurations")
sections.insert(idx + 1, new_section)
```

17-section flagship "Reading":

```
idx = next(i for i, s in enumerate(sections) if s["id"] == "astrology")
sections.insert(idx + 1, new_section)
```

```
```
```

The chart dict must include `asc` and the seven traditional planets  
(sun, moon, mercury, venus, mars, jupiter, saturn). Outer planets are  
not used as house rulers in this technique; if you pass them, they're  
ignored. Each planet is shape:

```
```python
```

```
{"sign": "Pisces", "deg": 24.87, "house": 7, "retrograde": False}
```

```
```
```

**\*\*Smoke-tested\*\*** against Jordan Ross Atkins's chart (verified data from  
`astrology\_readings\_cache`): emits 14 paragraphs (intro + 12 houses +  
closing), all classical-rulership correct (1st&10th→Mercury, 2nd&9th  
→Venus, 3rd&8th→Mars, 4th&7th→Jupiter, 5th&6th→Saturn, 11th→Moon,  
12th→Sun). 2nd-house read for Jordan: cusp Libra, lord Venus in detriment

in 8th, money runs through depth/shared resource – matches the read I walked him through manually.

## ## Add-on 2 – Five Follow-Up Questions

After a client receives their reading, they can ask up to 5 clarifying questions. Each is answered by Claude using the reading's full text as context, in the same voice, grounded in the placements already established. The 5-question ceiling is hard-enforced at the database level so cost is predictable and the follow-up surface can't cannibalize the reading.

### \*\*Files:\*\*

- `follow\_up\_questions.sql` – Supabase migration. Adds `public.reading\_followups` table with polymorphic FK to either `readings\_cache` or `astrology\_readings\_cache`, CHECK that `question\_index` is in 1..5, UNIQUE constraint preventing duplicate slots, RLS scaffold, indexes.
- `follow\_up\_prompt.md` – the Claude prompt template (system + user) plus the API contract and UI sketch.

### \*\*Apply the migration:\*\*

```
``sql
-- Via Supabase MCP:
mcp_supabase__apply_migration({
 project_id: 'tuayjblcswcmtuhoucv',
 name: 'reading_followups_v1',
 query: <contents of follow_up_questions.sql>
})
```

```
-- Or via Supabase CLI:
supabase db push
``
```

### \*\*Wire-up – Next.js route handler (sketch):\*\*

```
``ts
// app/api/readings/[id]/followups/route.ts
import { Anthropic } from '@anthropic-ai/sdk';
import { renderFollowUpPrompt } from '@lib/numen/followup-prompt';

export async function POST(req, { params }) {
 const { question_index, question } = await req.json();
 const reading_id = params.id;

 // 1. auth check: this client owns this reading
 // 2. validate question_index 1..5, not already used
 // 3. load reading paragraphs + structured chart
 const { reading, chart } = await loadReading(reading_id);

 // 4. render prompts
 const { system, user } = renderFollowUpPrompt({
 reading, chart, question, question_index
 });

 // 5. call Claude
 const anthropic = new Anthropic();
 const msg = await anthropic.messages.create({
 model: 'claude-opus-4-7',
 max_tokens: 1500,
 system,
 messages: [{ role: 'user', content: user }]
 });
}
```

```

});

const answer = msg.content.map(b => b.text).join('');

// 6. persist
await supabase.from('reading_followups').insert({
 [cacheTableForReading(reading_id)]: reading_id,
 seeker_name: chart.seeker_name,
 question_index,
 question,
 answer,
 composed_by: 'claude',
 claude_model: 'claude-opus-4-7',
 answered_at: new Date().toISOString(),
 token_input_count: msg.usage.input_tokens,
 token_output_count: msg.usage.output_tokens,
});

return Response.json({ answer });
}
...

```

**\*\*Wire-up – UI on the reading page:\*\***

```

```.tsx
// After the closing section in app/reading/[id]/page.tsx
<section className="reading-section followup-section">
  <h2>Five Follow-Up Questions</h2>
  <p className="kicker">
    Ask up to 5 questions to deepen this reading. Each is answered in
    the reading's own voice, grounded in the chart you were given.
  </p>
  <FollowUpQuestions readingId={reading.id} />
</section>
```

```

The `<FollowUpQuestions>` component manages the 0-5 state, POSTs to the route handler above, and renders the answer in the same prose style as the rest of the reading.

## ## Deploy order

1. Apply `follow_up_questions.sql` migration.
2. Drop `houses_and_lords.py` into the engine's composer module alongside the existing per-section composers.
3. Edit the section-composition pipeline so it appends `compose_houses_and_lords(chart)` after `configurations` (astrology product) and after `astrology` (flagship "Reading").
4. Add the Next.js route handler from the wire-up sketch above.
5. Add the `<FollowUpQuestions>` UI component.
6. Re-run the engine's composer for one test reading; verify the `houses-and-lords` section appears in the rendered output and the follow-up form posts cleanly.

## ## What this preserves

- Engine voice across both the new section and the follow-ups.
- No fabricated placements: deterministic composer for the houses section, reading-grounded Claude prompt for the follow-ups.
- Hard cost ceiling on follow-ups (5 max per reading, enforced in DB).
- Both products get the same upgrade.

## ## What still needs the engine repo

The engine code that composes sections, calls Claude, and inserts rows into the cache tables is not in this `numenist-site` repo – it lives in the separate Next.js Vercel app. To deploy these add-ons, the files above need to land in that repo. The deterministic composer and the SQL migration are self-contained; they can be dropped in without modification. The route handler and UI component need to be adapted to the engine repo's existing auth and Supabase client patterns.